

TourPlanner is a full-stack web application developed using ASP.NET Core (.NET 10) for the backend and Angular for the frontend. The purpose of the application is to manage tours, including their details, associated tour logs, and route information. Users can create, edit, delete, and search for tours, as well as view them on an interactive map.

The backend follows a 3-layer architecture. The Controller layer is responsible for handling HTTP requests from the frontend and returning responses. The Service layer contains the business logic, such as validating input data, handling authentication, and processing tour-related operations. The Repository layer manages all database operations using Entity Framework Core. This separation of concerns makes the application easier to understand, maintain, and extend.

The main entities in the system are Tour, TourLogs, TourCoordinates, and User. A tour contains basic information such as name, description, start and destination locations, and transport type. Each tour can also have multiple tour logs that store additional information like comments, difficulty, ratings, and distance. Each tour is linked to a user account, and authentication is handled using JWT tokens to ensure that only logged-in users can create or modify data.

A key feature of the project is the integration of the OpenRouteService API. When a tour is created, the route between the start and destination locations is automatically calculated. This route information is stored in the database and displayed on an interactive map in the frontend using Leaflet. This makes the application more visual and user-friendly.

The frontend is built using Angular with standalone components and communicates with the backend through REST APIs. It includes forms for creating and editing tours, views for displaying tour lists, and a map component for visualizing routes. State management is handled in a simple and reactive way using Angular Signals.

Unit testing was implemented using xUnit and Moq. The tests focus mainly on the business layer, especially validation logic in the TourService. Typical test cases include checking for missing fields, invalid input, and logical errors such as identical start and destination locations.

During development, one of the main lessons learned was the importance of a clean architecture with clearly separated layers. It also became clear that working with external APIs requires proper error handling and careful integration to avoid runtime issues.

<https://github.com/Beni1220/TourPlanner>](<https://github.com/Beni1220/TourPlanner>)